Step 1 — The Typed Shape and the Deterministic Validator — Solution

> Back to the step · Hint 1 · Hint 2

Story: the `Invoice` / `LineItem` models and the validator — the heart of the project — built and proven with no model at all. **If not:** if the validator is wrong, perfect extraction still produces data you can't trust.

Setup files

`requirements.txt`: `langchain-openai`, `langchain-community`, `openai`, `pydantic`, `python-dotenv` (add `pypdf` for `.pdf`). `.env.example`: `OPENAI_API_KEY=sk-your-key-here` / `LOG_LEVEL=INFO`. `.gitignore`: `.env`, `.venv/`, `__pycache__/`, `*.pyc`. Every folder under `src/` gets an empty `__init__.py`. `LICENSE` is the MIT text.

Settings, errors, logging, model

```
# src/exceptions.py
class MissingConfigError(Exception):
    """A required setting is missing from the environment."""
    pass

class UnsupportedFileError(Exception):
    """The invoice file type is not supported."""
    pass
```

```
# src/config.py
import os
from dataclasses import dataclass
from dotenv import load_dotenv
from src.exceptions import MissingConfigError

@dataclass
class Config:
    openai_api_key: str
    log_level: str
```

```
def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value

def load_config() -> Config:
    load_dotenv()
    return Config(
        openai_api_key=require_env("OPENAI_API_KEY"),
        log_level=os.getenv("LOG_LEVEL", "INFO"),
    )
```

```
# src/utils/logger.py
import logging
import os

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    if not logger.handlers:
        level_name = os.getenv("LOG_LEVEL", "INFO")
        level = getattr(logging, level_name.upper(), logging.INFO)
        handler = logging.StreamHandler()
        handler.setLevel(level)
        log_format = "%(asctime)s %(name)s %(levelname)s %(message)s"
        handler.setFormatter(logging.Formatter(log_format))
        logger.addHandler(handler)
        logger.setLevel(level)
    return logger
```

```
# src/models/llm.py
from langchain_openai import ChatOpenAI

from src.config import Config

def get_model(config: Config) -> ChatOpenAI:
    return ChatOpenAI(
        model="gpt-4o-mini", temperature=0, api_key=config.openai_api_key
    )
```

The typed shape

```
# src/schemas.py
# The typed shape of an invoice. The model fills this in; the validator checks
# it. Fields are optional so a missing one becomes a VALIDATION problem, not an
# invented value.
from typing import List, Optional

from pydantic import BaseModel

class LineItem(BaseModel):
    description: str
    quantity: int
    unit_price: float

class Invoice(BaseModel):
    vendor: Optional[str] = None
    invoice_number: Optional[str] = None
    line_items: List[LineItem] = []
    total: Optional[float] = None
```

The deterministic validator (the heart of the project)

```
# src/validator.py
# The deterministic checks the model can't be trusted with: do the line items
# add up to the stated total, and are required fields present? Plain Python, so
# it's fully testable with hand-made Invoice objects - no model needed.
from src.schemas import Invoice

def compute_subtotal(invoice: Invoice) -> float:
    subtotal = 0.0
    for item in invoice.line_items:
        subtotal += item.quantity * item.unit_price
    return round(subtotal, 2)

def validate_invoice(invoice: Invoice) -> list:
    problems = []
    if not invoice.invoice_number:
        problems.append("missing invoice number")
    if not invoice.vendor:
        problems.append("missing vendor")
    if not invoice.line_items:
        problems.append("no line items found")
    if invoice.total is None:
```

```
        problems.append("missing total")
    else:
        computed = compute_subtotal(invoice)
        if abs(computed - invoice.total) > 0.01:
            problems.append(
                f"total mismatch: line items sum to {computed}, "
                f"invoice says {invoice.total}"
            )
    return problems
```

The tests (no model, no key)

```
# tests/unit/test_config.py
# Run from the repo root: python -m tests.unit.test_config
import os

from src.config import load_config, require_env
from src.exceptions import MissingConfigError

TEST_SETTING = "INVOICEIQ_TEST_SETTING"

def test_missing_setting() -> None:
    os.environ.pop(TEST_SETTING, None)
    result = "WRONG"
    try:
        require_env(TEST_SETTING)
    except MissingConfigError:
        result = "OK"
    print(f"case 1 (missing setting) -> {result}")

def test_config_loads() -> None:
    os.environ["OPENAI_API_KEY"] = "sk-test-123"
    config = load_config()
    result = "OK"
    if config.openai_api_key != "sk-test-123":
        result = "WRONG"
    print(f"case 2 (config loads)    -> {result}")

if __name__ == "__main__":
    test_missing_setting()
    test_config_loads()
```

```
# tests/unit/test_validator.py
# Run from the repo root: python -m tests.unit.test_validator
```

```
from src.schemas import Invoice, LineItem
from src.validator import compute_subtotal, validate_invoice

def good_invoice() -> Invoice:
    return Invoice(
        vendor="Acme", invoice_number="INV-1001",
        line_items=[
            LineItem(description="Widget", quantity=3, unit_price=10.0),
            LineItem(description="Bolt", quantity=5, unit_price=2.0),
        ],
        total=40.0,
    )

def has_problem(problems, text: str) -> bool:
    for problem in problems:
        if text in problem:
            return True
    return False

def test_clean_invoice_has_no_problems() -> None:
    result = "OK"
    if validate_invoice(good_invoice()) != []:
        result = "WRONG"
    print(f"case 1 (clean invoice -> no problems) -> {result}")

def test_total_mismatch_is_caught() -> None:
    invoice = good_invoice()
    invoice.total = 99.0
    result = "OK"
    if not has_problem(validate_invoice(invoice), "total mismatch"):
        result = "WRONG"
    print(f"case 2 (wrong total -> flagged)       -> {result}")

def test_missing_number_is_caught() -> None:
    invoice = good_invoice()
    invoice.invoice_number = None
    result = "OK"
    if "missing invoice number" not in validate_invoice(invoice):
        result = "WRONG"
    print(f"case 3 (missing number -> flagged)    -> {result}")

def test_subtotal_adds_up() -> None:
    result = "OK"
    if compute_subtotal(good_invoice()) != 40.0:
        result = "WRONG"
    print(f"case 4 (subtotal = 40.0)              -> {result}")
```

```
if __name__ == "__main__":
    test_clean_invoice_has_no_problems()
    test_total_mismatch_is_caught()
    test_missing_number_is_caught()
    test_subtotal_adds_up()
```

Expected output:

```
case 1 (missing setting) -> OK
case 2 (config loads)    -> OK
```

```
case 1 (clean invoice -> no problems) -> OK
case 2 (wrong total -> flagged)       -> OK
case 3 (missing number -> flagged)    -> OK
case 4 (subtotal = 40.0)              -> OK
```

The validator is done and proven before the model is anywhere in the picture — that's the right order, because the checks are the part you actually have to trust.